

IT1223 (P) - Database Management Systems (P)
Department of Physical Science
University of Vavuniya
Revision Work Sheet - 01

1. Create database 'revision_worksheet'.

CODING:

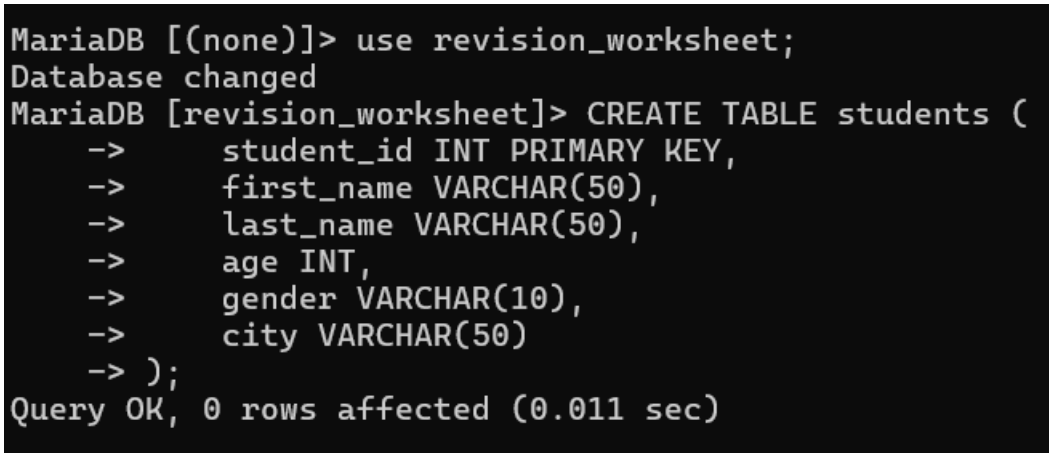
```
CREATE DATABASE revision_worksheet;  
USE revision_worksheet;
```

2. Create tables students, courses and enrollment with suitable datatypes.

CODING:

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    age INT,  
    gender VARCHAR(10),  
    city VARCHAR(50)  
);
```

OUTPUT:



```
MariaDB [(none)]> use revision_worksheet;  
Database changed  
MariaDB [revision_worksheet]> CREATE TABLE students (  
    ->     student_id INT PRIMARY KEY,  
    ->     first_name VARCHAR(50),  
    ->     last_name VARCHAR(50),  
    ->     age INT,  
    ->     gender VARCHAR(10),  
    ->     city VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.011 sec)
```

```
CREATE TABLE courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(100),  
    instructor VARCHAR(50),
```

```
credits INT,  
department VARCHAR(50)  
);
```

OUTPUT:

```
MariaDB [revision_worksheet]> CREATE TABLE courses (  
->     course_id INT PRIMARY KEY,  
->     course_name VARCHAR(100),  
->     instructor VARCHAR(50),  
->     credits INT,  
->     department VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.016 sec)
```

```
CREATE TABLE enrollment (  
    enrollment_id INT PRIMARY KEY,  
    student_id INT,  
    course_id INT,  
    enrollment_date DATE,  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id)  
);
```

OUTPUT:

```
MariaDB [revision_worksheet]> CREATE TABLE enrollment (  
->     enrollment_id INT PRIMARY KEY,  
->     student_id INT,  
->     course_id INT,  
->     enrollment_date DATE,  
->     FOREIGN KEY (student_id) REFERENCES students(student_id),  
->     FOREIGN KEY (course_id) REFERENCES courses(course_id)  
-> );  
Query OK, 0 rows affected (0.019 sec)
```

3. Write a SQL query to view the structure of the **students**, **courses** and **enrollment** tables.

CODING:

```
DESCRIBE students;
```

OUTPUT:

```
MariaDB [revision_worksheet]> DESCRIBE students;
```

Field	Type	Null	Key	Default	Extra
student_id	int(11)	NO	PRI	NULL	
first_name	varchar(50)	YES		NULL	
last_name	varchar(50)	YES		NULL	
age	int(11)	YES		NULL	
gender	varchar(10)	YES		NULL	
city	varchar(50)	YES		NULL	

```
6 rows in set (0.008 sec)
```

CODING:

```
DESCRIBE courses;
```

OUTPUT:

```
MariaDB [revision_worksheet]> DESCRIBE courses;
```

Field	Type	Null	Key	Default	Extra
course_id	int(11)	NO	PRI	NULL	
course_name	varchar(100)	YES		NULL	
instructor	varchar(50)	YES		NULL	
credits	int(11)	YES		NULL	
department	varchar(50)	YES		NULL	

```
5 rows in set (0.008 sec)
```

CODING:

```
DESCRIBE enrollment;
```

OUTPUT:

```
MariaDB [revision_worksheet]> DESCRIBE enrollment;
```

Field	Type	Null	Key	Default	Extra
enrollment_id	int(11)	NO	PRI	NULL	
student_id	int(11)	YES	MUL	NULL	
course_id	int(11)	YES	MUL	NULL	
enrollment_date	date	YES		NULL	

```
4 rows in set (0.006 sec)
```

- How to insert the values into table **students**, **courses** and **enrollment** tables.

CODING:

*** Inserting into students table**

```
INSERT INTO students (student_id, first_name, last_name, age, gender, city) VALUES
```

```
(1, 'John', 'Doe', 20, 'Male', 'New York'),
(2, 'Jane', 'Smith', 22, 'Female', 'Los Angeles'),
(3, 'Michael', 'Johnson', 21, 'Male', 'Chicago'),
(4, 'Emily', 'Brown', 19, 'Female', 'Houston'),
(5, 'David', 'Martinez', 23, 'Male', 'Miami'),
(6, 'Sarah', 'Jones', 20, 'Female', 'Boston'),
(7, 'Daniel', 'Garcia', 21, 'Male', 'Seattle'),
(8, 'Jennifer', 'Wilson', 22, 'Female', 'San Francisco'),
(9, 'Christopher', 'Anderson', 24, 'Male', 'Dallas'),
(10, 'Amanda', 'Thomas', 19, 'Female', 'Phoenix');
```

OUTPUT:

```
MariaDB [revision_worksheet]> INSERT INTO students (student_id, first_name,
last_name, age, gender, city) VALUES
-> (1, 'John', 'Doe', 20, 'Male', 'New York'),
-> (2, 'Jane', 'Smith', 22, 'Female', 'Los Angeles'),
-> (3, 'Michael', 'Johnson', 21, 'Male', 'Chicago'),
-> (4, 'Emily', 'Brown', 19, 'Female', 'Houston'),
-> (5, 'David', 'Martinez', 23, 'Male', 'Miami'),
-> (6, 'Sarah', 'Jones', 20, 'Female', 'Boston'),
-> (7, 'Daniel', 'Garcia', 21, 'Male', 'Seattle'),
-> (8, 'Jennifer', 'Wilson', 22, 'Female', 'San Francisco'),
-> (9, 'Christopher', 'Anderson', 24, 'Male', 'Dallas'),
-> (10, 'Amanda', 'Thomas', 19, 'Female', 'Phoenix');
Query OK, 10 rows affected (0.011 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

CODING:

*Inserting into courses table

```
INSERT INTO courses (course_id, course_name, instructor, credits, department) VALUES
(1, 'Introduction to Programming', 'Prof. Smith', 3, 'Computer Science'),
(2, 'Database Management', 'Prof. Johnson', 4, 'Computer Science'),
(3, 'Statistics', 'Prof. Brown', 3, 'Mathematics'),
(4, 'English Composition', 'Prof. Martinez', 3, 'English'),
(5, 'History of Art', 'Prof. Wilson', 3, 'Art'),
(6, 'Chemistry', 'Prof. White', 4, 'Science'),
(7, 'Economics', 'Prof. Garcia', 3, 'Social Science'),
(8, 'Algebra', 'Prof. Thompson', 3, 'Mathematics'),
(9, 'Psychology', 'Prof. Davis', 3, 'Social Science'),
(10, 'Physical Education', 'Prof. Lee', 2, 'Physical Education');
```

OUTPUT:

```
MariaDB [revision_worksheet]> INSERT INTO courses (course_id, course_name, instructor, credits, department) VALUES
-> (1, 'Introduction to Programming', 'Prof. Smith', 3, 'Computer Science'),
-> (2, 'Database Management', 'Prof. Johnson', 4, 'Computer Science'),
-> (3, 'Statistics', 'Prof. Brown', 3, 'Mathematics'),
-> (4, 'English Composition', 'Prof. Martinez', 3, 'English'),
-> (5, 'History of Art', 'Prof. Wilson', 3, 'Art'),
-> (6, 'Chemistry', 'Prof. White', 4, 'Science'),
-> (7, 'Economics', 'Prof. Garcia', 3, 'Social Science'),
-> (8, 'Algebra', 'Prof. Thompson', 3, 'Mathematics'),
-> (9, 'Psychology', 'Prof. Davis', 3, 'Social Science'),
-> (10, 'Physical Education', 'Prof. Lee', 2, 'Physical Education');
Query OK, 10 rows affected (0.002 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

CODING:

* Inserting into enrollment table

```
INSERT INTO enrollment (enrollment_id, student_id, course_id, enrollment_date) VALUES
(1, 1, 1, '2024-01-15'),
(2, 1, 3, '2024-01-20'),
(3, 2, 2, '2024-02-01'),
(4, 3, 4, '2024-02-10'),
(5, 4, 5, '2024-03-05'),
(6, 5, 6, '2024-03-10'),
(7, 6, 7, '2024-03-15'),
(8, 7, 8, '2024-04-01'),
(9, 8, 9, '2024-04-05'),
(10, 9, 10, '2024-04-10');
```

OUTPUT:

```
MariaDB [revision_worksheet]> INSERT INTO enrollment (enrollment_id, student_id, course_id, enrollment_date) VALUES
-> (1, 1, 1, '2024-01-15'),
-> (2, 1, 3, '2024-01-20'),
-> (3, 2, 2, '2024-02-01'),
-> (4, 3, 4, '2024-02-10'),
-> (5, 4, 5, '2024-03-05'),
-> (6, 5, 6, '2024-03-10'),
-> (7, 6, 7, '2024-03-15'),
-> (8, 7, 8, '2024-04-01'),
-> (9, 8, 9, '2024-04-05'),
-> (10, 9, 10, '2024-04-10');
Query OK, 10 rows affected (0.003 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

5. Retrieve the students details who are older than 20.

CODING:

```
SELECT * FROM students WHERE age > 20;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM students WHERE age > 20;
+-----+-----+-----+-----+-----+-----+
| student_id | first_name | last_name | age | gender | city |
+-----+-----+-----+-----+-----+-----+
| 2 | Jane | Smith | 22 | Female | Los Angeles |
| 3 | Michael | Johnson | 21 | Male | Chicago |
| 5 | David | Martinez | 23 | Male | Miami |
| 7 | Daniel | Garcia | 21 | Male | Seattle |
| 8 | Jennifer | Wilson | 22 | Female | San Francisco |
| 9 | Christopher | Anderson | 24 | Male | Dallas |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.001 sec)
```

6. Retrieve students who are older than 20 and from New York, or younger than 20 and from Los Angeles.

CODING:

```
SELECT * FROM students WHERE (age > 20 AND city = 'New York') OR (age < 20 AND city = 'Los Angeles');
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM students WHERE (age > 20 AND city = 'New York') OR (age < 20 AND city = 'Los Angeles');
Empty set (0.002 sec)
```

7. Retrieve students who are not from New York.

CODING:

```
SELECT * FROM students WHERE city <> 'New York';
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM students WHERE city <> 'New York';
```

student_id	first_name	last_name	age	gender	city
2	Jane	Smith	22	Female	Los Angeles
3	Michael	Johnson	21	Male	Chicago
4	Emily	Brown	19	Female	Houston
5	David	Martinez	23	Male	Miami
6	Sarah	Jones	20	Female	Boston
7	Daniel	Garcia	21	Male	Seattle
8	Jennifer	Wilson	22	Female	San Francisco
9	Christopher	Anderson	24	Male	Dallas
10	Amanda	Thomas	19	Female	Phoenix

```
9 rows in set (0.001 sec)
```

8. Write a SQL query to find the youngest student's age and oldest student's age.

CODING:

```
SELECT MIN(age) AS youngest_age, MAX(age) AS oldest_age FROM students;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT MIN(age) AS youngest_age, MAX(age) AS oldest_age FROM students;
```

youngest_age	oldest_age
19	24

```
1 row in set (0.001 sec)
```

9. Find the both the sum and the average of credits in the courses table using aggregate functions.

CODING:

```
SELECT SUM(credits) AS total_credits, AVG(credits) AS average_credits FROM courses;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT SUM(credits) AS total_credits, AVG(credits) AS average_credits FROM courses;
```

total_credits	average_credits
31	3.1000

```
1 row in set (0.001 sec)
```

10. Count the number of female students in the students table.

CODING:

```
SELECT COUNT(*) AS female_students_count FROM students WHERE gender = 'Female';
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT COUNT(*) AS female_students_count FROM students WHERE gender = 'Female';
+-----+
| female_students_count |
+-----+
| 5 |
+-----+
1 row in set (0.000 sec)
```

11. Retrieve the first 3 students from the students table.

CODING:

```
SELECT * FROM students LIMIT 3;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM students LIMIT 3;
+-----+-----+-----+-----+-----+-----+
| student_id | first_name | last_name | age | gender | city |
+-----+-----+-----+-----+-----+-----+
| 1 | John | Doe | 20 | Male | New York |
| 2 | Jane | Smith | 22 | Female | Los Angeles |
| 3 | Michael | Johnson | 21 | Male | Chicago |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)
```

12. Write a SQL query to find the students who are from New York and Los Angeles. (Hint: Use IN operator)

CODING:

```
SELECT * FROM students WHERE city IN ('New York', 'Los Angeles');
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM students WHERE city IN ('New York', 'Los Angeles');
+-----+-----+-----+-----+-----+-----+
| student_id | first_name | last_name | age | gender | city |
+-----+-----+-----+-----+-----+-----+
| 1 | John | Doe | 20 | Male | New York |
| 2 | Jane | Smith | 22 | Female | Los Angeles |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)
```

13. Retrieve all courses with a credit count between 3 and 4.

CODING:

```
SELECT * FROM courses WHERE credits BETWEEN 3 AND 4;
```


OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM courses WHERE credits BETWEEN 3 AND 4;
+-----+-----+-----+-----+-----+
| course_id | course_name          | instructor   | credits | department          |
+-----+-----+-----+-----+-----+
| 1         | Introduction to Programming | Prof. Smith  | 3       | Computer Science    |
| 2         | Database Management      | Prof. Johnson | 4       | Computer Science    |
| 3         | Statistics               | Prof. Brown  | 3       | Mathematics         |
| 4         | English Composition      | Prof. Martinez | 3       | English             |
| 5         | History of Art           | Prof. Wilson  | 3       | Art                 |
| 6         | Chemistry                | Prof. White   | 4       | Science             |
| 7         | Economics                | Prof. Garcia  | 3       | Social Science      |
| 8         | Algebra                  | Prof. Thompson | 3       | Mathematics         |
| 9         | Psychology               | Prof. Davis   | 3       | Social Science      |
+-----+-----+-----+-----+-----+
9 rows in set (0.001 sec)
```

14. Find the course details with more than 3 credits.

CODING:

```
SELECT * FROM courses WHERE credits > 3;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM courses WHERE credits > 3;
+-----+-----+-----+-----+-----+
| course_id | course_name          | instructor   | credits | department          |
+-----+-----+-----+-----+-----+
| 2         | Database Management  | Prof. Johnson | 4       | Computer Science    |
| 6         | Chemistry            | Prof. White   | 4       | Science             |
+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)
```

15. Write a SQL query to count the number of students in each city.

CODING:

```
SELECT city, COUNT(*) AS number_of_students FROM students GROUP BY city;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT city, COUNT(*) AS number_of_students FROM students GROUP BY city;
+-----+-----+
| city          | number_of_students |
+-----+-----+
| Boston       | 1                  |
| Chicago      | 1                  |
| Dallas       | 1                  |
| Houston      | 1                  |
| Los Angeles  | 1                  |
| Miami        | 1                  |
| New York     | 1                  |
| Phoenix      | 1                  |
| San Francisco | 1                  |
| Seattle      | 1                  |
+-----+-----+
10 rows in set (0.001 sec)
```

16. Find the cities that start with 'S'.

CODING:

```
SELECT DISTINCT city FROM students WHERE city LIKE 'S%';
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT DISTINCT city FROM students WHERE city
LIKE 'S%';
+-----+
| city          |
+-----+
| Seattle       |
| San Francisco |
+-----+
2 rows in set (0.001 sec)
```

17. Retrieve courses whose name contains 'Programming' or 'Management'.

CODING:

```
SELECT * FROM courses WHERE course_name LIKE '%Programming%' OR course_name
LIKE '%Management%';
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT * FROM courses WHERE course_name LIKE '%Programming%' OR course_name LIKE '%Management%';
+-----+-----+-----+-----+-----+
| course_id | course_name          | instructor | credits | department |
+-----+-----+-----+-----+-----+
| 1         | Introduction to Programming | Prof. Smith | 3       | Computer Science |
| 2         | Database Management      | Prof. Johnson | 4       | Computer Science |
+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)
```

18. How to retrieve students enrolled in courses using MYSQL.

CODING:

```
SELECT students.student_id, students.first_name, students.last_name, courses.course_name
FROM enrollment
JOIN students ON enrollment.student_id = students.student_id
JOIN courses ON enrollment.course_id = courses.course_id;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT students.student_id, students.first_name, students.last_name, courses.course_name
-> FROM enrollment
-> JOIN students ON enrollment.student_id = students.student_id
-> JOIN courses ON enrollment.course_id = courses.course_id;
```

student_id	first_name	last_name	course_name
1	John	Doe	Introduction to Programming
1	John	Doe	Statistics
2	Jane	Smith	Database Management
3	Michael	Johnson	English Composition
4	Emily	Brown	History of Art
5	David	Martinez	Chemistry
6	Sarah	Jones	Economics
7	Daniel	Garcia	Algebra
8	Jennifer	Wilson	Psychology
9	Christopher	Anderson	Physical Education

10 rows in set (0.001 sec)

19. Retrieve courses along with students enrolled in them.

CODING:

```
SELECT courses.course_id, courses.course_name, students.student_id, students.first_name,
students.last_name
FROM enrollment
JOIN courses ON enrollment.course_id = courses.course_id
JOIN students ON enrollment.student_id = students.student_id;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT courses.course_id, courses.course_name, students.student_id, students.first_name, students.last_name
-> FROM enrollment
-> JOIN courses ON enrollment.course_id = courses.course_id
-> JOIN students ON enrollment.student_id = students.student_id;
```

course_id	course_name	student_id	first_name	last_name
1	Introduction to Programming	1	John	Doe
3	Statistics	1	John	Doe
2	Database Management	2	Jane	Smith
4	English Composition	3	Michael	Johnson
5	History of Art	4	Emily	Brown
6	Chemistry	5	David	Martinez
7	Economics	6	Sarah	Jones
8	Algebra	7	Daniel	Garcia
9	Psychology	8	Jennifer	Wilson
10	Physical Education	9	Christopher	Anderson

10 rows in set (0.001 sec)

20. Retrieve students along with the courses they are enrolled in.

CODING:

```
SELECT students.student_id, students.first_name, students.last_name, courses.course_id,
courses.course_name
FROM enrollment
JOIN students ON enrollment.student_id = students.student_id
JOIN courses ON enrollment.course_id = courses.course_id;
```

OUTPUT:

```

MariaDB [revision_worksheet]> SELECT students.student_id, students.first_name, students.last_name, courses.course_id, courses.course_name
-> FROM enrollment
-> JOIN students ON enrollment.student_id = students.student_id
-> JOIN courses ON enrollment.course_id = courses.course_id;

```

student_id	first_name	last_name	course_id	course_name
1	John	Doe	1	Introduction to Programming
1	John	Doe	3	Statistics
2	Jane	Smith	2	Database Management
3	Michael	Johnson	4	English Composition
4	Emily	Brown	5	History of Art
5	David	Martinez	6	Chemistry
6	Sarah	Jones	7	Economics
7	Daniel	Garcia	8	Algebra
8	Jennifer	Wilson	9	Psychology
9	Christopher	Anderson	10	Physical Education

10 rows in set (0.001 sec)

21. Retrieve all possible combinations of students and courses.

CODING:

```

SELECT students.student_id, students.first_name, students.last_name, courses.course_id,
courses.course_name

FROM students

CROSS JOIN courses;

```

OUTPUT:

```

MariaDB [revision_worksheet]> SELECT students.student_id, students.first_name, students.last_name, courses.course_id, courses.course_name
-> FROM students
-> CROSS JOIN courses;

```

student_id	first_name	last_name	course_id	course_name
1	John	Doe	1	Introduction to Programming
2	Jane	Smith	1	Introduction to Programming
3	Michael	Johnson	1	Introduction to Programming
4	Emily	Brown	1	Introduction to Programming
5	David	Martinez	1	Introduction to Programming
6	Sarah	Jones	1	Introduction to Programming
7	Daniel	Garcia	1	Introduction to Programming
8	Jennifer	Wilson	1	Introduction to Programming
9	Christopher	Anderson	1	Introduction to Programming
10	Amanda	Thomas	1	Introduction to Programming
1	John	Doe	2	Database Management
2	Jane	Smith	2	Database Management
3	Michael	Johnson	2	Database Management
4	Emily	Brown	2	Database Management
5	David	Martinez	2	Database Management
6	Sarah	Jones	2	Database Management
7	Daniel	Garcia	2	Database Management
8	Jennifer	Wilson	2	Database Management
9	Christopher	Anderson	2	Database Management
10	Amanda	Thomas	2	Database Management
1	John	Doe	3	Statistics
2	Jane	Smith	3	Statistics
3	Michael	Johnson	3	Statistics
4	Emily	Brown	3	Statistics
5	David	Martinez	3	Statistics
6	Sarah	Jones	3	Statistics
7	Daniel	Garcia	3	Statistics
8	Jennifer	Wilson	3	Statistics
9	Christopher	Anderson	3	Statistics
10	Amanda	Thomas	3	Statistics
1	John	Doe	4	English Composition
2	Jane	Smith	4	English Composition
3	Michael	Johnson	4	English Composition
4	Emily	Brown	4	English Composition
5	David	Martinez	4	English Composition
6	Sarah	Jones	4	English Composition
7	Daniel	Garcia	4	English Composition
8	Jennifer	Wilson	4	English Composition
9	Christopher	Anderson	4	English Composition
10	Amanda	Thomas	4	English Composition

	1	John	Doe	4	English Composition
	2	Jane	Smith	4	English Composition
	3	Michael	Johnson	4	English Composition
	4	Emily	Brown	4	English Composition
	5	David	Martinez	4	English Composition
	6	Sarah	Jones	4	English Composition
	7	Daniel	Garcia	4	English Composition
	8	Jennifer	Wilson	4	English Composition
	9	Christopher	Anderson	4	English Composition
	10	Amanda	Thomas	4	English Composition
	1	John	Doe	5	History of Art
	2	Jane	Smith	5	History of Art
	3	Michael	Johnson	5	History of Art
	4	Emily	Brown	5	History of Art
	5	David	Martinez	5	History of Art
	6	Sarah	Jones	5	History of Art
	7	Daniel	Garcia	5	History of Art
	8	Jennifer	Wilson	5	History of Art
	9	Christopher	Anderson	5	History of Art
	10	Amanda	Thomas	5	History of Art
	1	John	Doe	6	Chemistry
	2	Jane	Smith	6	Chemistry
	3	Michael	Johnson	6	Chemistry
	4	Emily	Brown	6	Chemistry
	5	David	Martinez	6	Chemistry
	6	Sarah	Jones	6	Chemistry
	7	Daniel	Garcia	6	Chemistry
	8	Jennifer	Wilson	6	Chemistry
	9	Christopher	Anderson	6	Chemistry
	10	Amanda	Thomas	6	Chemistry
	1	John	Doe	7	Economics
	2	Jane	Smith	7	Economics
	3	Michael	Johnson	7	Economics
	4	Emily	Brown	7	Economics
	5	David	Martinez	7	Economics
	6	Sarah	Jones	7	Economics
	7	Daniel	Garcia	7	Economics
	8	Jennifer	Wilson	7	Economics
	9	Christopher	Anderson	7	Economics
	10	Amanda	Thomas	7	Economics
	1	John	Doe	8	Algebra
	2	Jane	Smith	8	Algebra
	3	Michael	Johnson	8	Algebra
	4	Emily	Brown	8	Algebra
	5	David	Martinez	8	Algebra
	6	Sarah	Jones	8	Algebra
	7	Daniel	Garcia	8	Algebra

7	Daniel	Garcia	7	Economics
8	Jennifer	Wilson	7	Economics
9	Christopher	Anderson	7	Economics
10	Amanda	Thomas	7	Economics
1	John	Doe	8	Algebra
2	Jane	Smith	8	Algebra
3	Michael	Johnson	8	Algebra
4	Emily	Brown	8	Algebra
5	David	Martinez	8	Algebra
6	Sarah	Jones	8	Algebra
7	Daniel	Garcia	8	Algebra
8	Jennifer	Wilson	8	Algebra
9	Christopher	Anderson	8	Algebra
10	Amanda	Thomas	8	Algebra
1	John	Doe	9	Psychology
2	Jane	Smith	9	Psychology
3	Michael	Johnson	9	Psychology
4	Emily	Brown	9	Psychology
5	David	Martinez	9	Psychology
6	Sarah	Jones	9	Psychology
7	Daniel	Garcia	9	Psychology
8	Jennifer	Wilson	9	Psychology
9	Christopher	Anderson	9	Psychology
10	Amanda	Thomas	9	Psychology
1	John	Doe	10	Physical Education
2	Jane	Smith	10	Physical Education
3	Michael	Johnson	10	Physical Education
4	Emily	Brown	10	Physical Education
5	David	Martinez	10	Physical Education
6	Sarah	Jones	10	Physical Education
7	Daniel	Garcia	10	Physical Education
8	Jennifer	Wilson	10	Physical Education
9	Christopher	Anderson	10	Physical Education
10	Amanda	Thomas	10	Physical Education

100 rows in set (0.001 sec)

22. Retrieve students who are enrolled in the same course.

CODING:

```
SELECT e1.student_id, e2.student_id AS other_student_id, e1.course_id
FROM enrollment e1
JOIN enrollment e2 ON e1.course_id = e2.course_id AND e1.student_id <> e2.student_id;
```

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT e1.student_id, e2.student_id AS other_student_id, e1.course_id
-> FROM enrollment e1
-> JOIN enrollment e2 ON e1.course_id = e2.course_id AND e1.student_id <> e2.student_id;
Empty set (0.001 sec)
```

23. Retrieve a combined list of first names from students and courses.

CODING:

```
SELECT first_name AS name FROM students
```

UNION

SELECT course_name AS name FROM courses;

OUTPUT:

```
MariaDB [revision_worksheet]> SELECT first_name AS name FROM studen
-> UNION
-> SELECT course_name AS name FROM courses;
+-----+
| name |
+-----+
| John |
| Jane |
| Michael |
| Emily |
| David |
| Sarah |
| Daniel |
| Jennifer |
| Christopher |
| Amanda |
| Introduction to Programming |
| Database Management |
| Statistics |
| English Composition |
| History of Art |
| Chemistry |
| Economics |
| Algebra |
| Psychology |
| Physical Education |
+-----+
20 rows in set (0.001 sec)
```